PROJECT REPORT

Domain-Specific RAG Chatbot

A Retrieval-Augmented Generation Chatbot utilizing TF-IDF and Gemini 3.5 Flash

1. Executive Summary

The **Domain-Specific RAG Chatbot** is a lightweight, efficient document question-answering application built with Python and Streamlit. It allows users to upload local files (PDF, DOCX, TXT) and conduct domain-specific inquiries. To ensure security, speed, and accuracy, the application restricts the language model's answers exclusively to the uploaded documents (Retrieval-Augmented Generation), preventing hallucinations and ensuring context-grounded citations.

2. System Architecture

The system employs a classic pipeline containing document ingestion, text preprocessing, vector-based chunk retrieval, prompt engineering, and LLM synthesis:

- **Ingestion**: Extracts plain text and maps it page-by-page from PDFs, DOCX, and TXT files using pypdf and python-docx.
- **Chunking & Overlap**: Divides long documents into chunks of 100 words. Introduces a sliding overlap window of 20 words to preserve context boundaries between chunks.
- **Retrieval Engine**: Converts the user's query and the parsed document chunks into numerical vectors using a local TF-IDF model. Computes Cosine Similarity scores to find the top 3 matching chunks, bypassing the cost and complexity of a external database.
- **LLM Generation**: Feeds the retrieved relevant chunks and the query into a prompt template, calling the Google Gemini API (gemini-3.5-flash) to formulate a strict, contextualized response.

3. Module Breakdown

3.1 app.py (User Interface)

Built with Streamlit. Manages UI elements (file upload sidebar, clear chat, process button), session states for conversation history, and displays cited sources dynamically with similarity scores.

3.2 document_loader.py (File Parser)

Contains functions that detect file extensions and extract clean text block arrays containing text, source filename, and page number metadata.

3.3 vector_store.py (Retrieval Engine)

Implements chunking logic and handles numerical vectorization and similarity matching using scikit-learn's TfidfVectorizer and cosine_similarity.

3.4 rag_pipeline.py & prompt.py (LLM Orchestration)

Integrates with the google-genai SDK and constructs structural prompts restricting the LLM to context facts only. If information is missing, the LLM is explicitly trained to declare: *'I could not find this information in the uploaded documents.'*

4. Technology Stack & Dependencies

Technology / Library	Version / Description	Role in Project
Streamlit	latest	Frontend UI & state handling
google-genai	latest	SDK for Gemini 3.5 Flash calls
scikit-learn	latest	TF-IDF vectorizer & cosine similarity
pypdf / python-docx	latest	File parsing and text extraction
python-dotenv	latest	Environment configuration (.env)

5. Installation & Setup Instructions

```
# 1. Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate

# 2. Install dependencies
pip install -r requirements.txt

# 3. Set API Key in .env file
GEMINI_API_KEY=your_gemini_api_key_here

# 4. Launch the application
streamlit run app.py
```